

软件开发指南

更新日期： 2026年06月15日

版本号：V0.0.1

目录

1 本体监控协议

- 1.1 协议总则
 - 1.1.1 协议层次
 - 1.1.2 协议端口号
 - 1.1.3 交互机制
 - 1.1.4 应用协议数据单元
 - 1.1.5 协议头部结构
 - 1.1.6 ASDU结构
- 1.2 控制类ASDU消息集
 - 1.2.1 心跳指令
 - 1.2.2 使用模式切换
 - 1.2.3 运动状态转换
 - 1.2.4 运动步态切换
 - 1.2.5 运动控制（轴指令）
 - 1.2.6 真实轴指令
 - 1.2.7 自定义灯语
 - 1.2.8 运动SDK模式下发
- 1.3 状态类ASDU消息集
 - 1.3.1 获取实时状态
 - 1.3.1.1 基础状态上报
 - 1.3.1.2 运控状态上报
 - 1.3.1.3 设备状态上报
 - 1.3.1.4 异常状态上报
- 1.4 巡检类ASDU消息集
 - 1.4.1 初始化和重置定位
 - 1.4.2 获取地图坐标系下的位置信息
 - 1.4.3 获取导航状态
 - 1.4.4 查询导航任务执行状态
- 1.5 通用协议接口调用状态响应ASDU消息集

2 ROS2话题

- 2.1 传感器驱动话题 -----
- 2.2 运动相关话题 -----
 - 2.2.1 运动状态转换 -----
 - 2.2.2 运控步态切换 -----
 - 2.2.3 获取运动基本状态和步态 -----
- 2.3 速度相关话题 -----
 - 2.3.1 获取或下发导航速度指令 -----
- 2.4 自主充电相关话题 -----
- 2.5 聚合故障相关话题 -----

附录1 UDP示例代码 -----

1 本体监控协议

1.1 协议总则

本协议基于TCP或UDP协议（根据具体开发需要进行选择），适用于机器人本体与上位机（外部板卡或系统）通信使用。

1.1.1 协议层次

本协议在OSI七层体系中所属位置以及协议栈数据结构，如下表所示：

本协议	应用层（第7层）
TCP/IP协议或UDP/IP协议	传输层（第4层）
	网络层（第3层）

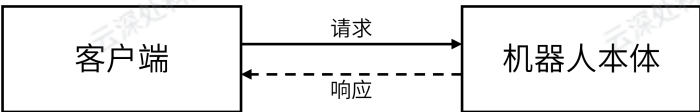
以太网	链路层（第2层）
	物理层（第1层）
注：第5，第6层未用	

1.1.2 协议端口号

本协议使用时，机器人本体为TCP/UDP服务端，外部板卡或系统为TCP/UDP客户端。DTLS 服务端（UDP + 加密）地址与端口为 10.21.33.103:30004，TLS 服务端（TCP + 加密）地址与端口为 10.21.33.103:30003。默认情况下，端口通信启用加密；用户可通过修改 r obotserve 配置文件关闭加密（需联系技术支持），且系统更新不会覆盖该配置文件。

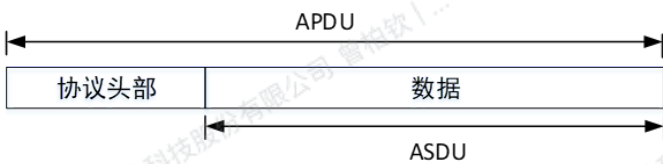
1.1.3 交互机制

如下图所示，本协议主要使用请求/响应机制。上位机可主动向机器人本体发起数据或状态查询请求，机器人本体通过响应回复部分请求。



1.1.4 应用协议数据单元

本协议APDU（应用协议数据单元）采用“协议头部+ASDU（应用服务数据单元）”结构，每个APDU可以携带1个ASDU。



1.1.5 协议头部结构

协议头部长固定16字节，内容如下表所示。

序号	内容	长度	值	备注
1	同步字符	1	0xeb	固定

2	同步字符	1	0x91	固定
3	同步字符	1	0xeb	固定
4	同步字符	1	0x90	固定
5	长度	2		APDU中ASDU字节段的长度，采用小端字节序，低字节在前，ASDU最大长度限制为65535字节
6	报文ID	2		每帧报文的唯一标识，用于识别请求帧和响应帧的对应关系，值由请求方控制，响应帧将采用相同值回复； 从0开始递增，达到65535后重新从0开始； 采用小端字节序，低字节在前
7	ASDU格式	1		ASDU数据格式类型判断位； ASDU数据内容采用XML格式时，值为 0x00； ASDU数据内容采用JSON格式时，值为 0x01
8	当前包编号	1	0x00	从0起始，顺序递增，最大255
9	协议版本号	1	0x01	固定
10	预留	5		预留5字节，暂填0

1.1.6 ASDU结构

本协议ASDU数据内容采用JSON或XML格式组织（参考“1.1.5 协议头部结构”部分，需在协议头部中写明），包含以下通用字段：

字段	含义
Type	消息类型
Command	消息命令码
Time	报文发送时间，本地时区，格式：YYYY-MM-DD HH:MM:SS
Items	消息所携带的参数项

【注意】推荐使用JSON格式，JSON格式下ASDU处理性能更快且数据结构更全面，支持更多消息类型。

一个JSON的ASDU案例如下：

▼ 复制代码

```
{
  "PatrolDevice": {
    "Type": 1002,
    "Command": 1,
    "Time": "2026-01-01 00:00:00",
    "Items": {
    }
  }
}
```

一个XML的ASDU案例如下：

▼ 复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>1002</Type>
  <Command>1</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items/>
</PatrolDevice>
```

1.2 控制类ASDU消息集

1.2.1 心跳指令

用户可通过该请求向机器人发送心跳指令。

Type字段值	Command字段值	消息类型
---------	------------	------

0x0010 0064	0x0000 0005	心跳指令
-------------	-------------	------

【注意】 建议以不小于1Hz的频率发送该指令，机器人会向持续发送心跳指令的IP和端口上报实时状态信息和异常状态信息，详见“1.3 状态类ASDU消息集”部分。

JSON格式请求：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0064,
    "Command": 0x0000 0005,
    "Time": "2026-01-01 00:00:00",
    "Items": {
    }
  }
}
```

XML格式请求：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0064</Type>
  <Command>0x0000 0005</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items/>
</PatrolDevice>
```

该请求消息中，Items字段不包含任何参数项。

1.2.2 使用模式切换

用户可通过该请求切换机器人使用模式，并参考“1.3 状态类ASDU消息集”部分中的反馈信息判断是否执行成功。

Type字段值	Command字段值	消息类型
0x0010 0002	0x0050 0002	使用模式切换

JSON格式请求：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0002,
    "Command": 0x0050 0002,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "Mode": 0, //0—常规模式，1—导航模式
    }
  }
}
```

XML格式请求：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0002</Type>
  <Command>0x0050 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <Mode>0</Mode>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段包含以下参数项：

参数项	含义	数据类型	单位	参数值说明
Mode	机器人使用模式	int	/	常规模式 = 0 导航模式 = 1 辅助模式 = 2

【注意】 常规模式下支持执行轴指令（见“1.2.5 运动控制”部分），导航模式下支持执行导航任务。

1.2.3 运动状态转换

用户可通过该请求转换机器人运动状态，并参考“1.3 状态类ASDU消息集”部分中的反馈信息判断是否执行成功。

Type字段值	Command字段值	消息类型
0x0010 0001	0x0020 0002	运动状态转换

JSON格式请求：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0001,
    "Command": 0x0020 0002,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "MotionParam": 0 //enum RobotMotionState
    }
  }
}
```

XML格式请求：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0001</Type>
  <Command>0x0020 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <MotionParam>0</MotionParam>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段包含以下参数项：

参数项	含义	数据类型	单位	参数值说明
-----	----	------	----	-------

MotionParam	机器人运动状态	int	/	软急停状态 = -2 默认值 = 0 站立状态= 1 关节阻尼状态 = 2 开机阻尼状态 = 3 趴下状态= 4 标零状态 = 5 小车移动状态 = 16 RL控制模式 = 17 阻尼趴下状态 = 0x100
				1

机器人运动状态

/

- 软急停状态 = -2
- 默认值 = 0
- 站立状态 = 1
- 关节阻尼状态 = 2
- 开机阻尼状态 = 3
- 趴下状态 = 4
- 标零状态 = 5
- 小车移动状态 = 16
- RL控制模式 = 17
- 阻尼趴下状态 = 0x100

【注意】“空闲”与“软急停”状态仅支持查询，不支持通过此命令下发转换。

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0001</Type>
  <Command>0x0030 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <GaitParam>4096</GaitParam>
    <ActionParam>12288</ActionParam>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段包含以下参数项：

参数项	含义	数据类型	单位	参数值说明
GaitParam	机器人运动步态	int	/	基础（标准运动模式） = 0x1001 楼梯（标准运动模式） = 0x1003 平地（导航运动模式） = 0x3002 楼梯（导航运动模式） = 0x3003

【注意】不同运动模式下支持不同的步态，步态切换时会自动切换到对应的运动模式；运动模式切换时，也会自动切换到对应的默认步态，运动模式请参考"1.2.3 运动状态转换"部分。

1.2.5 运动控制（轴指令）

用户可通过该请求控制机器人运动。

Type字段值	Command字段值	消息类型
0x0010 0001	0x0010 0002	运动控制

【注意】该指令仅支持在常规模式和辅助模式下执行。

JSON格式请求：

复制代码

```
{
```

```
"PatrolDevice": {
  "Type": 0x0010 0001,
  "Command": 0x0010 0002,
  "Time": "2026-01-01 00:00:00",
  "Items": {
    "X": 0.0,
    "Y": 0.0,
    "Z": 0.0,
    "Roll": 0.0,
    "Pitch": 0.0,
    "Yaw": 0.0
  }
}
```

XML格式请求:

▼ [复制代码](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0001</Type>
  <Command>0x0010 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <X>0.0</X>
    <Y>0.0</Y>
    <Z>0.0</Z>
    <Roll>0.0</Roll>
    <Pitch>0.0</Pitch>
    <Yaw>0.0</Yaw>
  </Items>
</PatrolDevice>
```

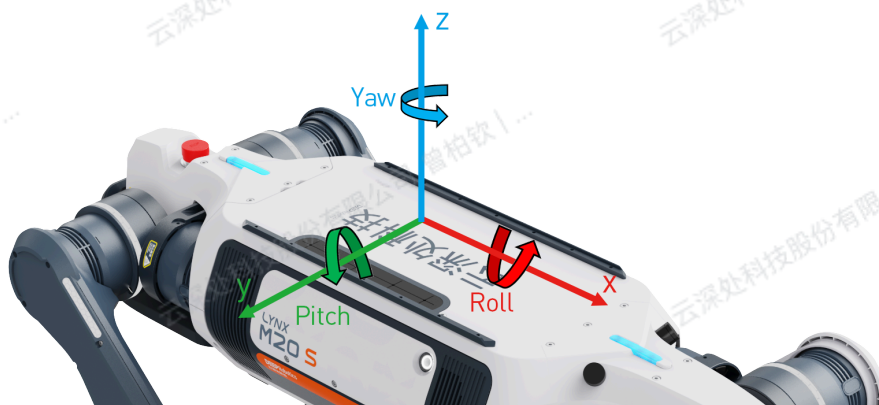
该请求消息中, Items字段包含以下参数项:

参数项	含义	数据类型	单位	参数值说明
X	前后方向运动速度	float	%	[-1,1] ([-1,1]之间的值表示当前指令速度相对于最大速度的比例, 1、-1表示以最大速度前进、后退)
Y	左右方向运动速度	float	%	[-1,1] ([-1,1]之间的值表示当前指令速度相对于最大速度的比例, 1、-1表示以最大速度左移、右移)

Z	高度方向运动速度	float	%	$[-1, 1]$ ($[-1, 1]$ 之间的值表示当前指令速度相对于最大速度的比例, 1、-1表示以最大速度向上、向下运动)
Roll	翻滚角	float	%	$[-1, 1]$ ($[-1, 1]$ 之间的值表示当前指令速度相对于最大速度的比例, 参数值的正负方向请参考右手坐标系确定)
Pitch	俯仰角	float	%	$[-1, 1]$ ($[-1, 1]$ 之间的值表示当前指令速度相对于最大速度的比例, 参数值的正负方向请参考右手坐标系确定)
Yaw	偏航角	float	%	$[-1, 1]$ ($[-1, 1]$ 之间的值表示当前指令速度相对于最大速度的比例, 参数值的正负方向请参考右手坐标系确定)

【注意】 仅在特殊步态下，六轴指令（X、Y、Z、Roll、Pitch、Yaw）全部有效；基础步态仅响应X、Y、Yaw三个自由度。

机器人的移动、旋转正方向请参考下图：



1.2.6 真实轴指令

用户可通过该请求下发真实轴控制指令。

Type字段值	Command字段值	消息类型
0x0010 0001	0x0011 0002	真实轴控制

【注意】 本指令的参数项与“1.2.5 运动控制（轴指令）”相同。不同之处在于，App下发的数据会原样传递给执行端，不会做额外的速度限制。

【注意】 本指令仅在导航模式下生效。

JSON格式请求：

▼复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0001,
    "Command": 0x0011 0002,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "X": 0.0,
      "Y": 0.0,
      "Z": 0.0,
      "Roll": 0.0,
      "Pitch": 0.0,
      "Yaw": 0.0
    }
  }
}
```

XML格式请求：

▼复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0001</Type>
  <Command>0x0011 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <X>0.0</X>
    <Y>0.0</Y>
    <Z>0.0</Z>
    <Roll>0.0</Roll>
    <Pitch>0.0</Pitch>
    <Yaw>0.0</Yaw>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段包含以下参数项：

参数项	含义	数据类型	单位
X	前后方向运动速度	float	m/s
Y	左右方向运动速度	float	m/s
Z	高度方向运动速度	float	m/s

Roll	翻滚角	float	raw/s
Pitch	俯仰角	float	raw/s
Yaw	偏航角	float	raw/s

1.2.7 自定义灯语

用户可通过该请求分别设置头灯和尾灯的灯光模式、颜色以及占空比。

Type字段值	Command字段值	消息类型
0x0010 0005	0x0020 0002	机器人灯语

JSON格式请求:

 复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0005,
    "Command": 0x0020 0002,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "CustomMode": true
      "Led" [{
        "Type": 0,
        "Color": [1],
        "Cycle": 1
      }, {
        "Type": 0,
        "Color": [2],
        "Cycle": 1
      }]
    }
  }
}
```

XML格式请求:

 复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<PatrolDevice>
  <Type>0x0010 0005</Type>
  <Command>0x0020 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <CustomMode>true</CustomMode>
    <Led>
      <Type>0</Type>
      <Color>1</Color>
      <Cycle>1</Cycle>
    </Led>
    <Led>
      <Type>0</Type>
      <Color>2</Color>
      <Cycle>1</Cycle>
    </Led>
  </Items>
</PatrolDevice>

```

该请求消息中，Items字段包含以下参数项：

参数项		含义	数据类型	单位	参数值说明
CustomMode		自定义灯语模式	bool	/	开启自定义模式 = ☐ true 关闭自定义模式 = ☐ false
Led [0]: 头灯 [1]: 尾灯	Type	灯光模式	int	/	常亮 = 0 填充式流水灯 = 1 移动式流水灯 = 2 渐变式流水灯 = 3 呼吸灯 = 4 单色闪烁灯 = 5
	Color	灯光颜色	array<int>	/	黑色 = 0 白色 = 1 绿灯 = 2 蓝灯 = 3
	Cycle	占空比	int	s	只有在呼唤和闪烁下才会使用占空比

1.2.8 运动SDK模式下发

用户可通过该请求下发运动SDK模式参数，控制机器人进入SDK运动控制模式。

Type字段值	Command字段值	消息类型
0x0010 0005	0x0030 0002	运动SDK模式下发

JSON格式请求：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0005,
    "Command": 0x0030 0002,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "SDKEnable": true,
      "Frequency": 100
    }
  }
}
```

XML格式请求：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0005</Type>
  <Command>0x0030 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <SDKEnable>true</SDKEnable>
    <Frequency>100</Frequency>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段包含以下参数项：

参数项	含义	数据类型	参数值说明
SDKEnable	SDK 模式使能	bool	开启运动SDK模式 = true 关闭运动SDK模式 = false

Frequency	关节话题数据发出频率	int	取值范围：[1, 200]且能整除1000
-----------	------------	-----	-----------------------

1.3 状态类ASDU消息集

1.3.1 获取实时状态

机器人会向发送心跳指令（参考“1.2.1 心跳指令”）的IP地址和端口主动上报状态信息。

1.3.1.1 基础状态上报

该消息集以2Hz的频率主动上报，用户可通过该上报获取机器人当前的基础状态信息。

Type字段值	Command字段值	消息类型
0x0010 0064	0x00f0 0000	获取机器人基础状态信息

JSON格式响应：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0064,
    "Command": 0x00f0 0000,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "BasicStatus": {
        "MotionState": 0,
        "Gait": 0,
        "HES": 0,
        "ControlUsageMode": 0,
        "PowerManagement": 0,
        "Sleep": false,
        "Model": "CD1",
        "Version": "STD",
```

```
        "Charge":0
    }
}
```

XML格式响应：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0064</Type>
  <Command>0x00f0 0000</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <BasicStatus>
      <MotionState>0</MotionState>
      <Gait>0</Gait>
      <HES>0</HES>
      <ControlUsageMode>0</ControlUsageMode>
      <PowerManagement>0</PowerManagement>
      <Sleep>>false</Sleep>
      <Model>CA9C</Model>
      <Version>STD</Version>
      <Charge>0</Charge>
    </BasicStatus>
  </Items>
</PatrolDevice>
```

Items字段中的部分参数项含义如下：

参数项	含义	数据类型	单位	参数值说明
-----	----	------	----	-------

Charge	充电状态	int	/	空闲 = 0x0000 前往电桩过程中 = 0x0001 充电中 = 0x0002 退出电桩过程中 = 0x0003 机器人异常 = 0x0004 在桩上，但未充电 = 0x0005 线充 = 0x0006
--------	------	-----	---	---

1.3.1.2 运控状态上报

该消息类型以10Hz的频率主动上报，用户可通过该消息类型的上报获取机器人当前的运动控制状态信息。

Type字段值	Command字段值	消息类型
0x0010 0001	0x00f0 0000	获取机器人运控状态信息

JSON格式响应：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0001,
    "Command": 0x00f0 0000,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "MotionStatus": {
        "Body": {
          "Roll": 0,
          "Pitch": 0,
          "Yaw": 0,
          "OmegaZ": 0
        },
        "Leg": {
          "Roll": 0,
          "Pitch": 0,
          "Yaw": 0,
          "OmegaZ": 0
        },
        "LinearX": 0,
```

XML格式响应:

 复制代码

```

    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    <Joint>0</Joint>
    </MotorStatus>
  </Items>
</PatrolDevice>

```

该响应消息中，Items字段包含MotionStatus和MotorStatus两组参数。

MotionStatus参数组反馈机器人整机的运动状态信息：

参数项	含义	数据类型	单位
Body.Roll	机器人身体的姿态角度	float	rad
Body.Pitch			
Body.Yaw			
Body.OmegaZ	身体Z方向角速度	float	rad/s
Leg.Roll	机器人腿部的姿态角度	float	rad
Leg.Pitch			
Leg.Yaw			
Leg.OmegaZ	腿部Z方向角速度	float	rad/s
LinearX/LinearY	机器人当前X / Y方向线速度	float	m/s
Height	当前机身高度	float	m
Payload	当前负载	float	kg
RemainMile	预测剩余里程	float	km

MotorStatus^[1]参数组反馈机器人各个关节的运动状态信息：


```
"DevEnable": {
  "Lidar": 0,
  "Video": 0,
},
"CPU": {
  "AOS": {
    "AvgUtil": 25,
    "CurFreqKhz": [
      1800000,
      1800000,
      1800000,
      1800000,
      2352000,
      2352000,
      2352000,
      2352000
    ],
    "GovPolicy": [
      "performance",
      "performance",
      "performance",
      "performance",
      "performance",
      "performance",
      "performance",
      "performance"
    ],
    "HwMaxFreqKhz": [
      1800000,
      1800000,
      1800000,
      1800000,
      2352000,
      2352000,
      2352000,
      2352000
    ],
    "HwMinFreqKhz": [
      408000,
      408000,
      408000,
      408000,
      408000,
      408000,
      408000,
      408000
    ]
  }
}
```

```

        408000
    ],
    "PackageTemp": 40,
    "SocId": "CPU_103",
    "Temps": [
        45,
        45,
        43,
        45,
        45,
        44,
        44,
        43
    ],
    "Timestamp": {
        "Nanosec": 716490114,
        "Sec": 1775813219
    },
    "Util": [
        30,
        39,
        28,
        37,
        27,
        9,
        19,
        11
    ]
},
"GOS": {
    // 同AOS结构, 仅SocId存在差异
},
"NOS": {
    // 同AOS结构, 仅SocId存在差异
}
}
,
}

```

XML格式响应:

复制代码

```

<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
    <Type>0x00100002</Type>
    <Command>0x00f00000</Command>

```

```
<Time>2026-01-01 00:00:00</Time>
<Items>
  <BatteryList>
    <BatteryItem>
      <Voltage>0.0</Voltage>
      <BatteryLevel>0.0</BatteryLevel>
      <battery_temperature>0.0</battery_temperature>
      <charge>>false</charge>
      <serial></serial>
    </BatteryItem>
  </BatteryList>
  <DeviceTemperature>
    <Motor>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
    </Motor>
    <Driver>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
      <value>0</value>
    </Driver>
  </DeviceTemperature>
</Items>
```

```
<value>0</value>
<value>0</value>
</Driver>
</DeviceTemperature>
<DevEnable>
  <Lidar>0</Lidar>
  <Video>0</Video>
</DevEnable>
<CPU>
  <AOS>
    <AvgUtil>25</AvgUtil>
    <CurFreqKhz>
      <freq>1800000</freq>
      <freq>1800000</freq>
      <freq>1800000</freq>
      <freq>1800000</freq>
      <freq>2352000</freq>
      <freq>2352000</freq>
      <freq>2352000</freq>
      <freq>2352000</freq>
    </CurFreqKhz>
    <GovPolicy>
      <policy>performance</policy>
      <policy>performance</policy>
      <policy>performance</policy>
      <policy>performance</policy>
      <policy>performance</policy>
      <policy>performance</policy>
      <policy>performance</policy>
      <policy>performance</policy>
    </GovPolicy>
    <HwMaxFreqKhz>
      <freq>1800000</freq>
      <freq>1800000</freq>
      <freq>1800000</freq>
      <freq>1800000</freq>
      <freq>2352000</freq>
      <freq>2352000</freq>
      <freq>2352000</freq>
      <freq>2352000</freq>
    </HwMaxFreqKhz>
    <HwMinFreqKhz>
      <freq>408000</freq>
      <freq>408000</freq>
      <freq>408000</freq>
```

```

        <freq>408000</freq>
        <freq>408000</freq>
        <freq>408000</freq>
        <freq>408000</freq>
        <freq>408000</freq>
    </HwMinFreqKhz>
    <PackageTemp>40</PackageTemp>
    <SocId>CPU_103</SocId>
    <Temps>
        <temp>45</temp>
        <temp>45</temp>
        <temp>43</temp>
        <temp>45</temp>
        <temp>45</temp>
        <temp>44</temp>
        <temp>44</temp>
        <temp>43</temp>
    </Temps>
    <Timestamp>
        <Nanosec>716490114</Nanosec>
        <Sec>1775813219</Sec>
    </Timestamp>
    <Util>
        <util>30</util>
        <util>39</util>
        <util>28</util>
        <util>37</util>
        <util>27</util>
        <util>9</util>
        <util>19</util>
        <util>11</util>
    </Util>
</AOS>
<GOS>
    <!-- 同AOS结构，仅SocId存在差异 -->
</GOS>
<NOS>
    <!-- 同AOS结构，仅SocId存在差异 -->
</NOS>
</CPU>

```

该响应消息中，Items字段包含BatteryList、DeviceTemperature、DevEnable和CPU四组参数。

BatteryList[0] 参数组用于反馈电池的状态信息，该参数组中的每个参数项包含以下子项：

参数子项	含义	数据类型	单位	参数值说明
Voltage	电池电压	float	V	
BatteryLevel	电池剩余电量百分比	int	%	取值范围：[0, 100]
battery_temperature	电池温度	float	°C	
charge	电池充电状态	bool	/	充电中 = ☐ true 未在充电 = ☐ false
serial	电池序列号	string	/	

DeviceTemperature 参数组用于反馈机器人各关节的电机温度及驱动器温度信息：

参数项	含义	数据类型	单位
Motor	电机温度列表	array<float>	°C
Driver	电机驱动器温度列表	array<float>	°C

DevEnable 参数组用于反馈雷达、相机等外设的使能状态以及电源控制状态：

参数项	含义	数据类型	单位	参数值说明
Lidar	雷达使能状态	int	/	关闭 = ☐ 0 开启 = ☐ 1 开启中 = ☐ 2
Video	相机开关	int	/	关闭 = ☐ 0 开启 = ☐ 1

CPU 参数组用于反馈运动主机（AOS）、导航主机（NOS）和通用主机（GOS）的处理器运行状态：

参数项	含义	数据类型	单位	参数值说明
-----	----	------	----	-------


```

"Time": "2026-01-01 00:00:00",
"Items": {
  "ErrorList": [{
    "Code": 32783,
    "Details": "",
    "Grouped": false,
    "Name": "joint_position_over_limit",
    "Resources": ["11", "13"],
    "Severities": [4, 4],
    "Source": ["r1_deploy", "r1_deploy"],
    "SourceIds": [11, 13],
    "Timestamp": {
      "Nanosec": 969825624,
      "Sec": 1774507557
    },
    "Type": 1
  }, {
    "Code": 32789,
    "Details": "",
    "Grouped": false,
    "Name": "joint_data_update_error",
    "Resources": ["11", "1"],
    "Severities": [4, 4],
    "Source": ["r1_deploy", "r1_deploy"],
    "SourceIds": [11, 1],
    "Timestamp": {
      "Nanosec": 793819587,
      "Sec": 1774507546
    },
    "Type": 1
  }]
}

```

XML格式响应:



复制代码

```

<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 007f</Type>
  <Command>0x00f0 0000</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <ErrorList>
      <Code>32783</Code>

```

```
<Details></Details>
<Grouped>>false</Grouped>
<Name>joint_position_over_limit</Name>
<Resources>
  <Resource>11</Resource>
  <Resource>13</Resource>
</Resources>
<Severities>
  <Severity>4</Severity>
  <Severity>4</Severity>
</Severities>
<Source>
  <SourceItem>r1_deploy</SourceItem>
  <SourceItem>r1_deploy</SourceItem>
</Source>
<SourceIds>
  <SourceId>11</SourceId>
  <SourceId>13</SourceId>
</SourceIds>
<Timestamp>
  <Nanosec>969825624</Nanosec>
  <Sec>1774507557</Sec>
</Timestamp>
<Type>1</Type>
</ErrorList>
<ErrorList>
  <Code>32789</Code>
  <Details></Details>
  <Grouped>>false</Grouped>
  <Name>joint_data_update_error</Name>
  <Resources>
    <Resource>11</Resource>
    <Resource>1</Resource>
  </Resources>
  <Severities>
    <Severity>4</Severity>
    <Severity>4</Severity>
  </Severities>
  <Source>
    <SourceItem>r1_deploy</SourceItem>
    <SourceItem>r1_deploy</SourceItem>
  </Source>
  <SourceIds>
    <SourceId>11</SourceId>
    <SourceId>1</SourceId>
```

```

</SourceIds>
<Timestamp>
  <Nanosec>793819587</Nanosec>
  <Sec>1774507546</Sec>
</Timestamp>
<Type>1</Type>
</ErrorList>
</Items>

```

一次响应可能包含多组ErrorList，每组ErrorList包含以下参数项：

参数项	含义	类型	参数值说明
Code	故障类型的唯一标识符	int	同一类故障（如"关节超限"）拥有相同的编码，不同类故障编码互不相同。完整映射关系请参见错误码表
Details	故障补充说明	string	
Grouped	合并模式	bool	多资源组跟踪模式（同类故障可聚合多个部件） = true 单资源独立跟踪模式（同类故障仅上报单个部件） = false
Name	故障名称	string	
Resources	发生错误的部件位置信息	string[]	例如：11表示第11个关节在心跳超时事件中显示故障模块
Severities	故障对应的严重等级	int[]	WARN = 3 ERROR = 4 FATAL = 5
Source	故障来源模块标识	string[]	
SourceIds	故障来源实例编号	int[]	
Timestamp	故障发生时间戳	object	
+Nanosec	纳秒级时间	int	例如：969825624表示第969825624毫秒
+Sec	秒级时间	int	例如：1774507557表示第1774507557秒

Type	故障事件类型	int	TYPE_START1（故障首次发生或严重等级发生变化） = 1 TYPE_STOP（故障已消除） = 2
------	--------	-----	--

Code的值对应的含义如下表所示：

参数值	含义	参数值	含义
0x8101	电池电量低	0x8005	驱动器电流过大
0x8102	电池电压过低	0x8006	驱动器通信超时
0x8104	单体电池欠压	0x8007	驱动器状态错误
0x8105	单体电池过压	0x8008	驱动器运行异常
0x8106	整组电池过压	0x800A	关节持续高速运转
0x8107	整组电池欠压	0x8012	温感线断联
0x8108	电池温度过高	0x8014	关节位置超限
0x8109	电池温度过低	0x8016	关节数据无效
0x810A	放电电流过大	0x8017	关节数据更新错误
0x810D	充电电流过大	0x8301	CPU温度过高
0x810E	输出短路	0x8302	CPU使用率过高
0x810F	IC错误	0x8303	CPU主频过低
0x8110	MOS锁定	0x8304	可用内存过低
0x8111	数据采集失败	0x8306	文件系统使用率过高
0x8201	IMU发布超时/无更新	0x8308	inode使用率过高
0x8202	IMU打开失败	0x9401	获取反光柱定位数据超时
0x8203	IMU数据超时（关节级）	0x9402	反光柱定位算法开启失败
0x8204	IMU数据无效	0x9403	无反光柱定位信息
0x8001	电机温度过高	0x9406	无电流检测

0x8002	驱动器温度过高	0x9408	退桩遇到障碍物
0x8003	驱动器电压过高	0x9409	到达充电站超时
0x8004	驱动器电压过低		

1.4 巡检类ASDU消息集

1.4.1 初始化和重置定位

用户可通过该请求初始化机器人定位。

Type字段值	Command字段值	消息类型
0x0010 0003	0x0020 0002	初始化和重置定位

JSON格式请求：

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0020 0002,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "PosX": 0.0,
      "PosY": 0.0,
      "PosZ": 0.0,
      "Yaw": 0.0
    }
  }
}
```

XML格式请求：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
  <Command>0x0020 0002</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <PosX>0.0</PosX>
    <PosY>0.0</PosY>
    <PosZ>0.0</PosZ>
    <Yaw>0.0</Yaw>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段包含以下参数项：

参数项	含义	数据类型	单位
PosX	机器人在地图坐标系下需要重新定位的X轴坐标	float	m
PosY	机器人在地图坐标系下需要重新定位的Y轴坐标	float	m
PosZ	机器人在地图坐标系下需要重新定位的Z轴坐标	float	m
Yaw	机器人在地图坐标系下绕Z轴旋转的姿态角	float	rad

1.4.2 获取地图坐标系下的位置信息

用户可通过该请求获取机器人当前在地图坐标系下的位置信息。

Type字段值	Command字段值	消息类型
0x0010 0003	0x0020 0001	获取地图坐标系下机器人位置信息

请求

JSON格式请求：



复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0020 0001,
    "Time": "2026-01-01 00:00:00",
    "Items": {
    }
  }
}
```

XML格式请求:

▼ [复制代码](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
  <Command>0x0020 0001</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
  </Items>
</PatrolDevice>
```

该请求消息中, Items字段不包含任何参数项。

响应

JSON格式响应:

▼ [复制代码](#)

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0020 0001,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "Location": 1,
      "PosX": 0.0,
      "PosY": 0.0,
      "PosZ": 0.0,
      "Roll": 0.0,
      "Pitch": 0.0,
      "Yaw": 0.0
    }
  }
}
```

XML格式响应：

▼

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
  <Command>0x0020 0001</Command>
  <Time>2026-01-01 00:00:01</Time>
  <Items>
    <Location>1</Location>
    <PosX>0.0</PosX>
    <PosY>0.0</PosY>
    <PosZ>0.0</PosZ>
    <Roll>0.0</Roll>
    <Pitch>0.0</Pitch>
    <Yaw>0.0</Yaw>
  </Items>
</PatrolDevice>
```

Items字段中的参数项含义如下：

参数项	含义	数据类型	单位	参数值说明
Location	定位状态	int	/	定位正常 = 0 定位丢失 = 1
PosX/PosY/PosZ	机器人在地图坐标系下的坐标	float	m	
Roll/Pitch/Yaw	机器人在地图坐标系下的姿态角度	float	rad	

1.4.3 获取导航状态

用户可通过该请求获取机器人导航中的感知软件状态信息。

Type字段值	Command字段值	消息类型
0x0010 0003	0x0011 0001	获取导航中的感知软件状态信息

请求

JSON格式请求：

▼

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0011 0001,
    "Time": "2026-01-01 00:00:00",
    "Items": {
    }
  }
}
```

XML格式请求：

▼

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
  <Command>0x0011 0001</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
  </Items>
</PatrolDevice>
```

该请求消息中，Items字段不包含任何参数项。

响应

JSON格式响应：

▼

复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0011 0001,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "Location": 1,
      "ObsState": 0.0
    }
  }
}
```

XML格式响应：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
  <Command>0x0011 0001</Command>
  <Time>2026-01-01 00:00:01</Time>
  <Items>
    <Location>1</Location>
    <ObsState>0.0</ObsState>
  </Items>
</PatrolDevice>
```

Items字段中的参数项含义如下：

参数项	含义	数据类型	单位	参数值说明
Location	定位状态	int	/	定位正常 = 0 定位丢失 = 1
ObsState	是否处于避障状态	int	/	无障碍物，未处于避障状态 = 0 有障碍物，处于避障状态 = 1

1.4.4 查询导航任务执行状态

用户可通过该请求查询导航任务执行状态。此外，当导航任务发生异常时，系统将通过 Toast 提示或弹窗向用户报告异常状态，以使用户了解异常原因并采取相应的处理措施。

Type字段值	Command字段值	消息类型
0x0010 0003	0x0010 0001	查询导航任务执行状态

请求

JSON格式请求：

复制代码

```
{
```

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0010 0001,
    "Time": "2026-01-01 00:00:00",
    "Items": {
    }
  }
}
```

XML格式请求:

▼ 复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
  <Command>0x0010 0001</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items/>
</PatrolDevice>
```

该请求消息中, Items字段不包含任何参数项。

响应

JSON格式响应:

▼ 复制代码

```
{
  "PatrolDevice": {
    "Type": 0x0010 0003,
    "Command": 0x0010 0001,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "Value": 0,
      "Status": 0,
      "ErrorCode": 0
    }
  }
}
```

XML格式响应:

▼ 复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>0x0010 0003</Type>
```

```
<Command>0x0010 0001</Command>
<Time>2026-01-01 00:00:01</Time>
<Items>
  <Value>0</Value>
  <Status>0</Status>
  <ErrorCode>0</ErrorCode>
</Items>
</PatrolDevice>
```

Items字段中的参数项含义如下：

参数项	含义	数据类型	单位	参数值说明
Value	导航任务目标点编号	int	/	与下发导航任务请求对应
Status	导航任务执行状态	int	/	空闲 = 0 退出充电桩中 = 1 导航预处理 = 2 导航中 = 3 导航完成 = 4 进入充电桩中 = 5 暂停中 = 0xff
ErrorCode	错误码	int	/	详见导航错误码表

ErrorCode的值对应的含义如下表所示：

参数项	含义	参数项	含义
0x0000	导航任务正常状态	0xA34C	局部导航规划失败
0x2300	导航任务正常结束	0xA34D	持续导航速度未刷新，导航失败
0x2302	主动取消导航任务	0xA34E	自主充电流程中，下发任务失败
0x8605	移动到点超时	0xA400	发送目标点到局部导航失败

0xA30 1	运动状态异常	0xA401	发送目标点到全局导航失败
0xA30 2	低电量或电池模式	0xA402	取消局部导航失败
0xA30 3	电机状态异常	0xA403	取消全局导航失败
0xA30 5	雷达异常（未开启或无点云数据）	0xA404	导航参数设置失败
0xA31 2	Planner非空闲状态或者全局导航状态异常	0xA405	局部导航模块异常，已重启
0xA31 3	定位异常	0xA406	局部导航模块异常：无实时障碍地图
0xA31 4	地形图异常	0xA407	局部导航模块异常：无定位数据
0xA32 7	步态异常，未切换到目标步态	0xA408	全局导航模块异常，已重启
0xA32 8	当前不在导航模式，无法执行导航任务	0xA409	全局导航模块异常：无栅格地图
0xA34 1	导航任务进行中，下发任务异常	0xA40A	全局导航模块异常：无定位数据
0xA34 3	等待退出自主充电超时	0xA40B	全局导航模块异常：全局导航规划失败
0xA34 4	自主充电异常	0xA40C	全局导航模块异常：目标点在障碍物中
0xA34 5	自主充电执行失败	0xA40D	定位模块异常，已重启
0xA34 9	切换地图失败	0xA40E	定位模块异常：传感器异常
0xA34 B	局部导航-持续避障	0xA40F	里程计模块异常：里程计结果发布时间异常

【注意】若查询发现导航任务无法执行，请检查导航程序是否未启动或存在异常。

1.5 通用协议接口调用状态响应ASDU消息集

该响应消息集与请求的消息集一一对应，在消息请求后自动响应。用户可通过该响应获取协议接口的调用状态。

JSON格式响应：

复制代码

```
{
  "PatrolDevice": {
    "Type": 1007,
    "Command": 1,
    "Time": "2026-01-01 00:00:00",
    "Items": {
      "ErrorCode": 0,
      "ErrorMessage": "Success"
    }
  }
}
```

XML格式响应：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<PatrolDevice>
  <Type>1007</Type>
  <Command>1</Command>
  <Time>2026-01-01 00:00:00</Time>
  <Items>
    <ErrorCode>0</ErrorCode>
    <ErrorMessage>Success</ErrorMessage>
  </Items>
</PatrolDevice>
```

Type字段值

Command字段值

消息类型

与请求的消息类型相同

与请求的消息命令码相同

通用协议接口调用状态响应

【注意】对于本章"1 本体监控协议"中的协议，协议请求下发后，返回的通用协议接口调用状态响应消息集与发送的请求消息集一一对应；且响应消息中的 `Type` 和 `Command` 字段值与对应请求消息中的 `Type` 和 `Command` 字段值相同。

Items字段中参数项的值与含义如下（ErrorMessage与ErrorCode一一对应，用作对ErrorCode的补充描述）：

ErrorCode 值	ErrorMessage值	ErrorMessage含 义	补充说明
0x0000	Success	成功	
0xE001	Unsupported data format	数据格式不支持	发送的数据格式不支持（非XML 和JSON）
0xE002	Data parsing failed	数据解析失败	协议格式异常，缺少 PatrolDevice、Items、Type、 Command
0xE003	Unsupported protocol type	不支持的协议	找不到Type、Command对应的 协议
0xE004	Missing necessary fields	缺少必要字段	缺少协议字段
0xE005	Mismatched data type	字段类型不匹配	
0xE006	Mismatched request client	请求客户端不匹配	例如：轴指令发送时，要求2s内 发送的指令来自同一个客户端
0xE007	No operation permission	无操作权限	
0xE008	Not allowed operation	不允许的操作	
0xE009	Operation failure	操作失败	
0xE00A	Unsupported function	不支持的功能	例如：导航功能只有Pro版本支 持
0xE00B	Internal error	内部错误	例如：内部文件打开失败，内部 文件解析失败

【注意】其他非通用ErrorCode错误码请参考对应指令章节中的具体说明。

2 ROS2话题

本节介绍山猫S10机器人的ROS2话题，部分话题采用自定义消息类型。

【注意】使用ROS2 工具接收话题、查看频率等前，需要 `source /opt/robot/scripts/setup_ros2.sh` 环境变量。

【注意】建议使用ROS2 工具接收话题、查看频率等前，通过 `su` 获取管理员权限，以确保相关指令可正常执行。

2.1 传感器驱动话题

该节的话题均为开机后即有消息发布，开发者可订阅话题消息，以获取传感器数据。

话题名	描述	消息类型	频率
/IMU	通用格式IMU数据话题	sensor_msgs/msg/Imu	200Hz
/LIDAR/POINTS	通用格式点云数据话题	sensor_msgs/msg/PointCloud2	10Hz

【注意】/LIDAR/POINTS 话题依赖于 `multicast-relay.service` 服务，若话题无数据，可参考附录2查看服务的存活状态，并开启、关闭相应服务。

【注意】/LIDAR/POINTS 话题必须要 `su` 获取管理员权限之后才能获取到数据。

【注意】/LIDAR/POINTS 话题被限制为仅机器人主机传输，如需外接请联系技术支持。

2.2 运动相关话题

【注意】 本节话题依赖于 `rl_deploy` 服务，若话题消息下发无响应或收不到话题消息，可参考附录2查看服务的存活状态，并开启、关闭相应服务。

2.2.1 运动状态转换

该话题在运动程序启动后激活，开发者可在该话题中发布消息，以转换机器人运动状态。

话题名	描述	消息类型
<code>/MOTION_STATE</code>	运动状态 ^[1] 转换指令	<code>drdds/msg/MotionState</code>

话题 `/MOTION_STATE` 的消息类型 `drdds/msg/MotionState` 是自定义消息类型，包含 `drdds::msg::MetaType` 和 `drdds::msg::MotionStateValue`：

▼

复制代码

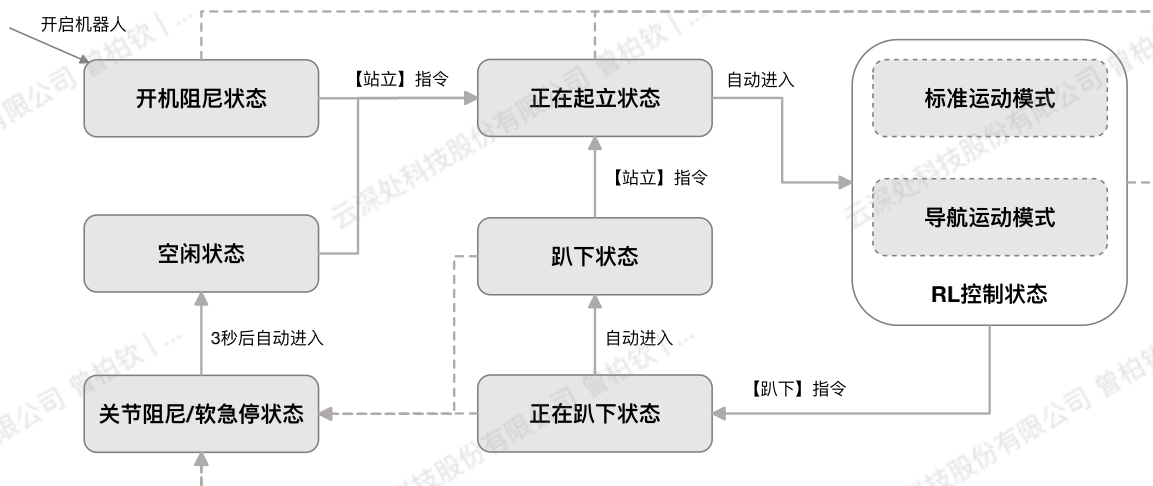
```
MetaType header
  uint64 frame_id
  Timestamp timestamp
    int32 sec
    uint32 nsec
MotionStateValue data
  int32 state
```

设置不同的运动状态对应需要发布的话题消息字段 `state` 如下表所示：

需要设置的运动状态	应发布的话题消息
空闲	0
站立	1
关节阻尼/软急停	2
开机阻尼	3
趴下	4
RL控制	17

【注意】 “空闲”与“开机阻尼”状态非下发值，仅作为只读状态。

[1] 机器人运动状态转换关系如下图所示：



【注意】 完成“站立”后，需尽快切换至“RL控制”。RL控制状态下，分为标准运动模式和敏捷运动模式：标准运动模式适用于手动控制；敏捷运动模式下的步态速度响应性能较好，适用于导航等自主算法开发。

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

2.2.2 运控步态切换

该话题在运动程序启动后激活，开发者可在该话题中发布消息，以切换运控步态。

话题名	描述	消息类型
/GAIT	运控步态切换指令	drdds/msg/Gait

话题 `/GAIT` 的消息类型 `drdds/msg/Gait` 是自定义消息类型，包含 `drdds::msg::MetaType` 和 `drdds::msg::GaitValue`：

复制代码

```
MetaType header
uint64 frame_id
Timestamp timestamp
int32 sec
uint32 nsec
GaitValue data
```

设置不同的运动状态对应需要发布的话题消息字段 `gait` 如下表所示:

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

该话题在开机后即有消息发布，开发者可订阅话题消息，以获取运动基本状态和步态。

话题 `/MOTION_INFO` 的消息类型 `drdds/msg/MotionInfo` 是自定义消息类型，包含 `drdds::msg::MetaType`、`drdds::msg::MotionInfoValue`、`drdds::msg::MotionStateValue` 和 `drdds::msg::GaitValue`：

```

MetaType header
    uint64 frame_id
    Timestamp timestamp
    int32 sec
    uint32 nsec
MotionInfoValue data
    float32 vel_x
    float32 vel_y
    float32 vel_yaw
    float32 height
MotionStateValue data
    int32 state
GaitValue data
    uint32 gait
    float32 payload

```

50 / 58

字段名	字段含义	数据类型	单位	字段值说明
vel_x	前后方向运动速度	float32	m/s	正、负值分别表示前进、后退速度
vel_y	左右方向运动速度	float32	m/s	正、负值分别表示左移、右移速度
vel_yaw	水平转向速度	float32	rad/s	正、负值分别表示逆时针、顺时针旋转速度
height	当前机身高度	float32	m	
state	运动状态	int32	/	空闲 = 0 站立 = 1 关节阻尼/软急停 = 2 开机阻尼 = 3 趴下 = 4 RL控制 = 17
gait	步态	uint32	/	基础（标准运动模式） = 0x1001 平地（敏捷运动模式） = 0x3002 楼梯（敏捷运动模式） = 0x3003
payload	无效参数	/	/	

【注意】 标准运动模式适用于手动控制；敏捷运动模式下的步态速度响应性能较好，适用于导航等自主算法开发。

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

2.3 速度相关话题

2.3.1 获取或下发导航速度指令

【注意】发布该话题时，可能会与机器人自带的 `planner` 服务产生冲突，请在参考附录2关闭 `planner` 服务。同时，该话题依赖 `basic_server` 服务，若速度指令无响应，请参考附录2查看服务的存活状态，并开启、关闭相应服务。此外，发布该话题时，可能会与机器人自带的自主充电程序所使用的 `charge_manager` 服务产生冲突，请避免在机器人执行自主充电任务时发布该话题，或参考附录2关闭 `charge_manager` 服务以禁用自主充电功能。

【注意】该速度话题仅在导航模式下生效，且建议以10Hz的固定频率下发该话题。

【注意】开发过程中，如无需使用机器人自带的定位服务，可参考附录2关闭 `localization` 服务以节省资源。

开发者可通过发布该话题来下发导航规划的速度指令，以控制机器人移动；也可订阅话题消息，以获机器人本身导航功能下发的速度指令。

话题名	描述	消息类型
/NAV_CMD	获取或下发导航规划的速度指令	drdds::msg::NavCmd

话题 `/NAV_CMD` 的消息类型 `drdds/msg/NavCmd` 是自定义消息类型，包含 `drdds::msg::MetaType` 和 `drdds::msg::NavCmdValue`：

复制代码

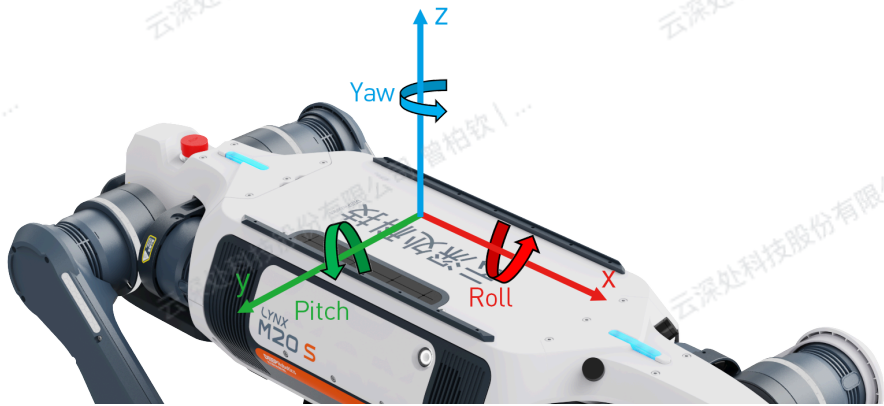
```
MetaType header
uint64 frame_id
Timestamp timestamp
int32 sec
uint32 nsec
NavCmdValue data
float32 x_vel
float32 y_vel
float32 yaw_vel
```

消息内容对应的字段含义如下表所示：

字段	含义	数据类型	单位	字段值说明
x_vel	前后方向运动速度	float32	m/s	正、负值分别表示前进、后退速度
y_vel	左右方向运动速度	float32	m/s	正、负值分别表示左移、右移速度

yaw_ve 	偏航角速度	float32	rad/ s	正、负值分别表示逆时针、顺时针旋转速度
------------	-------	---------	-----------	---------------------

机器人的移动、旋转正方向请参考下图：



开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

2.4 自主充电相关话题

开发者可通过订阅该话题来获取机器人的自主充电状态，以便实时监控充电过程；也可获取发生异常时的错误码，以便诊断故障原因。

话题名	描述	消息类型
/CHARGE_STATUS	自主充电状态	drdds/msg/StdStatus

话题 `/CHARGE_STATUS` 的消息类型 `drdds/msg/StdStatus` 为自定义消息类型：

✓

复制代码

```
int32 state
uint32 error_code
```

话题 `/CHARGE_STATUS` 的消息内容对应的自主充电状态如下表所示：

字段名	字段含义	数据类型	字段值	对应的自主充电状态	对应的异常状态
-----	------	------	-----	-----------	---------

state	自主充电状态	int32	0	空闲	/
			1	前往电桩过程中	/
			2	充电中	/
			3	退出电桩过程中	/
			4	机器人异常	0x1002 ~ 0x1008
			5	在桩上但无电流	0x1007
error_code	异常状态码	uint32	0	空闲	重置操作 / 初始化
			1	空闲 / 充电中	操作成功
			4098 (0x1002)	机器人异常	寻找充电桩目标点超时 (反光柱遮挡超时)
			4099 (0x1003)	机器人异常	开启反光柱定位算法失败
			4100 (0x1004)	机器人异常	寻找充电桩目标点失败 (无反光柱目标点)
			4101 (0x1005)	机器人异常	进桩超时
			4102 (0x1006)	机器人异常	退桩超时
			4103 (0x1007)	机器人异常	充电桩无电流
			4104 (0x1008)	机器人异常	发生软急停

2.5 聚合故障相关话题

开发者可通过订阅该话题来获取机器人聚合后的故障快照（发布频率约为1Hz），从而实时了解当前系统中所有的活跃故障。

话题名	描述	消息类型
/fault_aggregator	聚合故障快照	drdds/msg/FaultEventArray

话题 `/fault_aggregator` 的消息类型 `drdds/msg/FaultEventArray` 为自定义消息类型，其包含一个时间戳 `timestamp` 和一个 `FaultEvent` 数组 `active_faults`，分别表示快照生成时间和当前活跃的故障事件列表：

▼	复制代码
<pre>builtin_interfaces/Time timestamp diagnostic_manager/FaultEvent[] active_faults</pre>	

其中，一个 `FaultEvent` 字段解读示例如下：

▼	复制代码
<pre>timestamp: {sec: 1707312000, nanosec: 0} name: "motor_over_temp" code: 32769 type: 1 = STOP(异常恢复) severities: [3] = FATAL resources: ["joint_0"] grouped: false source: ["motion_master#0"] (motion_master, SOC0) details: '{"temp":85.0}' 业务模块自行定义，无固定Schema</pre>	
	← 故障发生时间 ← 故障名称 ← 0x8001 故障码 ← 1 = START(异常开始), 2 = STOP(异常恢复) ← 3 = WARN, 4 = ERROR, 5 = FATAL ← 故障资源（关节0） ← 单资源模式 ← 来源进程 ← 附加信息（可选），格式由业务模块自行定义，无固定Schema

故障码定义如下：

参数值	含义	参数值	含义
0x8101	电池电量低	0x8005	驱动器电流过大
0x8102	电池电压过低	0x8006	驱动器通信超时

0x8104	单体电池欠压	0x8007	驱动器状态错误
0x8105	单体电池过压	0x8008	驱动器运行异常
0x8106	整组电池过压	0x800A	关节持续高速运转
0x8107	整组电池欠压	0x8012	温感线断联
0x8108	电池温度过高	0x8014	关节位置超限
0x8109	电池温度过低	0x8016	关节数据无效
0x810A	放电电流过大	0x8017	关节数据更新错误
0x810D	充电电流过大	0x8301	CPU温度过高
0x810E	输出短路	0x8302	CPU使用率过高
0x810F	IC错误	0x8303	CPU主频过低
0x8110	MOS锁定	0x8304	可用内存过低
0x8111	数据采集失败	0x8306	文件系统使用率过高
0x8201	IMU发布超时/无更新	0x8308	inode使用率过高
0x8202	IMU打开失败	0x9401	获取反光柱定位数据超时
0x8203	IMU数据超时（关节级）	0x9402	反光柱定位算法开启失败
0x8204	IMU数据无效	0x9403	无反光柱定位信息
0x8001	电机温度过高	0x9406	无电流检测
0x8002	驱动器温度过高	0x9408	退桩遇到障碍物
0x8003	驱动器电压过高	0x9409	到达充电站超时
0x8004	驱动器电压过低		

【注意】 此表为常见故障速查，完整定义以实际配置文件 `fault_rules.toml` 和 `common.hpp` 中的 `FaultCode` 枚举为准。

附录1 UDP示例代码

▼ 发送心跳指令

复制代码

```
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>

#define SERVER_IP "10.21.31.103"
#define PORT 30000
#define BUFFER_SIZE 1024

struct udpMessage
{
    unsigned char header[16];
    unsigned char data[BUFFER_SIZE];
};

int main() {
    // 创建 UDP 套接字
    int client_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (client_fd < 0) {
        perror("socket creation failed");
        return -1;
    }

    // 配置服务器地址
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr) <= 0) {
        perror("Invalid address/ Address not supported");
        close(client_fd);
        return -1;
    }

    udpMessage message;
    message.header[0] = 0xeb;
    message.header[1] = 0x91;
    message.header[2] = 0xeb;
```

```

message.header[3] = 0x90;

// 要发送的 JSON 字符串
const char *data = R"(
{
    "PatrolDevice": {
        "Type": 100,
        "Command": 100,
        "Time": "2026-01-01 00:00:00",
        "Items": {
        }
    }
}
)";

unsigned short dataLength = strlen(data);

message.header[4] = dataLength & 0xFF;
message.header[5] = (dataLength >> 8) & 0xFF;
message.header[6] = 0x01;
message.header[7] = 0x00;
message.header[8] = 0x01;

memcpy(message.data, data, strlen(data));

// 发送数据
ssize_t send_len = sendto(client_fd, &message, dataLength + 16, 0,
(struct sockaddr *)&server_addr, sizeof(server_addr));
if (send_len < 0) {
    perror("sendto failed");
    close(client_fd);
    return -1;
}

std::cout << "Message sent successfully." << std::endl;

// 关闭套接字
close(client_fd);
return 0;
}

```